

```
// File: src/App.vue
<script setup>
import { RouterView } from 'vue-router'
import ToastContainer from './components/ToastContainer.vue'
import ErrorBoundary from './components/ErrorBoundary.vue'
import PWAUpdatePrompt from './components/PWAUpdatePrompt.vue'
</script>
```

```
<template>
  <div class="app-shell">
    <ErrorBoundary>
      <RouterView />
    </ErrorBoundary>
    <ToastContainer />
    <PWAUpdatePrompt />
  </div>
</template>
```

```
<style scoped>
.app-shell {
  min-height: 100vh;
}
</style>
```

```
// File: src/main.js
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'
import './style.css'
```

```
const app = createApp(App)
```

```
app.use(router)
app.mount('#app')
```

```
// File: src/router.js
import { createRouter, createWebHistory } from 'vue-router'
```

```
const routes = [
  {
    path: '/',
    name: 'Home',
    component: () => import('./pages/Home.vue')
  },
  {
    path: '/difficulty',
    name: 'DifficultySelect',
    component: () => import('./pages/DifficultySelect.vue')
  },
  {
```

```

    path: '/game/:id',
    name: 'Game',
    component: () => import('./pages/Game.vue'),
    props: true
  },
  {
    path: '/explore',
    name: 'NumberExplore',
    component: () => import('./pages/NumberExplore.vue'),
    meta: { title: '数字探索' }
  },
  // 404 - 未匹配的路由重定向到首页
  {
    path: '/*',
    redirect: '/'
  }
]

const router = createRouter({
  history: createWebHistory(),
  routes,
  scrollBehavior() {
    return { left: 0, top: 0 }
  }
})

export default router

// File: src/pages/DifficultySelect.vue
<script setup>
import { computed, nextTick, onMounted, ref } from 'vue'
import { useRouter } from 'vue-router'
import { ArrowLeft, Trophy } from 'lucide-vue-next'
import { DIFFICULTY_GROUPS, getDifficultyById, TOTAL_LEVELS } from '../config/difficulty'
import { useStorage } from '../composables/useStorage'
import { useSound } from '../composables/useSound'
import DifficultyCard from '../components/DifficultyCard.vue'

const router = useRouter()
const { getBestScore, completedIds, getLeaderboard } = useStorage()
const { playClick, playBack } = useSound()

const isLeaving = ref(false)
const BACK_NAVIGATION_DELAY = 180
const SCROLL_STORAGE_KEY = 'math-game-difficulty-scroll-y'

const completedCount = computed(() => completedIds.value.length)
const progressPercent = computed(() => Math.round((completedCount.value / TOTAL_LEVELS) * 100))

function getSavedScrollY() {

```

```
    try {
      return Number(sessionStorage.getItem(SCROLL_STORAGE_KEY))
    } catch (error) {
      console.error('读取关卡页滚动位置失败:', error)
      return 0
    }
  }
}

function saveScrollY() {
  try {
    sessionStorage.setItem(SCROLL_STORAGE_KEY, String(window.scrollY))
  } catch (error) {
    console.error('保存关卡页滚动位置失败:', error)
  }
}

function clearSavedScrollY() {
  try {
    sessionStorage.removeItem(SCROLL_STORAGE_KEY)
  } catch (error) {
    console.error('清理关卡页滚动位置失败:', error)
  }
}

onMounted(async () => {
  const savedScrollY = getSavedScrollY()

  if (!Number.isFinite(savedScrollY) || savedScrollY <= 0) {
    window.scrollTo(0, 0)
    return
  }

  await nextTick()
  requestAnimationFrame(() => {
    window.scrollTo(0, savedScrollY)
    clearSavedScrollY()
  })
})

function goBack(event) {
  if (isLeaving.value) {
    return
  }

  isLeaving.value = true
  clearSavedScrollY()
  playBack()
  const btn = event?.currentTarget
  if (btn) {
    btn.classList.add('is-leaving')
```

```
    }
    setTimeout(() => {
      router.push('/')
    }, BACK_NAVIGATION_DELAY)
  }

function selectDifficulty(event, difficulty) {
  if (isLeaving.value) {
    return
  }

  isLeaving.value = true
  saveScrollY()
  playClick()
  const btn = event?.currentTarget
  if (btn) {
    btn.classList.add('is-leaving')
  }
  router.push(`/game/${difficulty.id}`)
}

function getDifficultyBestScore(difficultyId) {
  return getBestScore(difficultyId)
}

function getDifficultyLeaderboard(difficultyId) {
  const difficulty = getDifficultyById(difficultyId)
  return getLeaderboard(difficultyId, difficulty.questionCount)
}

const lockedStatusMap = computed(() => {
  const map = new Map()
  const completed = completedIds.value

  DIFFICULTY_GROUPS.forEach((group) => {
    group.levels.forEach((id) => {
      if (id === 1) {
        map.set(id, false)
        return
      }

      map.set(id, !completed.includes(id - 1))
    })
  })

  return map
})

function isDifficultyLocked(difficulty) {
  return lockedStatusMap.value.get(difficulty.id) ?? false
}
```

```
}
</script>

<template>
  <div class="page">
    <header class="header-panel">
      <div class="nav-row">
        <button class="back-btn" aria-label="返回首页" @click="goBack($event)">
          <ArrowLeft :size="18" />
        </button>
        <div class="header-badge">
          <Trophy :size="16" />
          <span>{{ completedCount }}/{{ TOTAL_LEVELS }}</span>
        </div>
      </div>

      <div class="headline-row">
        <div>
          <p class="eyebrow">选一关</p>
          <h1 class="title text-child-2xl">从 1 开始</h1>
          <p class="subtitle text-child-sm">先玩上面的。</p>
        </div>
      </div>

      <div class="progress-panel">
        <div class="progress-copy">
          <span>已玩</span>
          <strong>{{ progressPercent }}%</strong>
        </div>
        <div class="progress-track">
          <div class="progress-fill" :style="{ width: `${progressPercent}%` }"></div>
        </div>
      </div>
    </header>

    <main class="main-content">
      <section
        v-for="group in DIFFICULTY_GROUPS"
        :key="group.name"
        class="section"
      >
        <div class="section-header">
          <span class="section-badge" :class="`badge-${group.color}`">{{ group.name }}</span>
        </div>

        <div class="card-list">
          <DifficultyCard
            v-for="id in group.levels"
            :key="id"
            :difficulty="getDifficultyById(id)"
          />
        </div>
      </section>
    </main>
  </div>
</template>
```

```
      :is-locked="isDifficultyLocked(getDifficultyById(id))"
      :is-completed="completedIds.includes(id)"
      :best-score="getDifficultyBestScore(id)"
      :leaderboard="getDifficultyLeaderboard(id)"
      @select="selectDifficulty"
    />
  </div>
</section>
</main>
</div>
</template>

<style scoped>
.page {
  min-height: 100vh;
  min-height: 100dvh;
  padding: calc(env(safe-area-inset-top, 0px) + 18px) 14px 36px;
}

.header-panel {
  position: sticky;
  top: calc(env(safe-area-inset-top, 0px) + 10px);
  z-index: 10;
  margin-bottom: 18px;
  padding: 18px;
  border-radius: var(--radius-xl);
  background: rgba(255, 255, 255, 0.94);
  border: 1px solid var(--border-light);
  box-shadow: var(--shadow-sm);
}

.nav-row,
.progress-copy,
.section-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.nav-row {
  margin-bottom: 18px;
}

.back-btn,
.header-badge {
  display: inline-flex;
  align-items: center;
  border-radius: 14px;
  font-weight: 700;
}
```

```
.back-btn {
  justify-content: center;
  width: 48px;
  height: 48px;
  color: var(--text-secondary);
  background: rgba(255, 255, 255, 0.92);
  border: 1px solid var(--border-light);
}

.header-badge {
  gap: 8px;
}

.back-btn:active {
  transform: scale(0.98);
}

.back-btn.is-leaving {
  opacity: 0.7;
  transform: scale(0.96);
  transition: all var(--duration-normal) var(--ease-out);
}

@media (hover: hover) {
  .back-btn:hover {
    color: var(--text-primary);
    background: rgba(255, 255, 255, 0.9);
  }
}

.header-badge {
  padding: 10px 12px;
  color: var(--candy-blue-dark);
  background: rgba(49, 120, 246, 0.08);
}

.headline-row {
  margin-bottom: 18px;
}

.eyebrow {
  margin-bottom: 8px;
  color: var(--candy-blue-dark);
  font-size: var(--font-sm);
  font-weight: 800;
  letter-spacing: 0.1em;
  text-transform: uppercase;
}
```

```
.title {
  color: var(--text-primary);
  font-weight: 800;
}

.subtitle {
  margin-top: 8px;
  color: var(--text-secondary);
  line-height: 1.7;
}

.progress-panel {
  padding: 16px;
  border-radius: var(--radius-lg);
  background: rgba(255, 255, 255, 0.9);
  border: 1px solid var(--border-light);
}

.progress-copy {
  margin-bottom: 10px;
  color: var(--text-secondary);
  font-size: var(--font-sm);
  font-weight: 700;
}

.progress-copy strong {
  color: var(--text-primary);
}

.progress-track {
  height: 10px;
  overflow: hidden;
  border-radius: var(--radius-full);
  background: rgba(49, 120, 246, 0.12);
}

.progress-fill {
  height: 100%;
  border-radius: inherit;
  background: linear-gradient(90deg, var(--candy-blue), var(--candy-mint));
}

.main-content {
  display: flex;
  flex-direction: column;
  gap: 18px;
}

.section {
  opacity: 1;
```



```
    transform: none;
  }

.section-header {
  margin-bottom: 8px;
  padding: 4px 2px;
}

.section-badge {
  display: inline-flex;
  align-items: center;
  width: fit-content;
  border-radius: 0;
  font-weight: 800;
  padding: 0;
  color: var(--text-muted);
  font-size: 12px;
}

.badge-mint {
  background: var(--candy-mint-soft);
  color: var(--candy-mint-dark);
}

.badge-pink {
  background: var(--candy-blue-soft);
  color: var(--candy-blue-dark);
}

.badge-yellow {
  background: var(--candy-yellow-soft);
  color: var(--candy-yellow-dark);
}

.badge-peach {
  background: var(--candy-peach-soft);
  color: var(--candy-peach-dark);
}

.badge-lavender {
  background: var(--candy-lavender-soft);
  color: var(--candy-lavender-dark);
}

.card-list {
  display: flex;
  flex-direction: column;
  gap: 12px;
}
```

```
@media (min-width: 768px) {  
  .page {  
    max-width: 980px;  
    margin: 0 auto;  
    padding: 24px 20px 40px;  
  }  
  
  .card-list {  
    display: grid;  
    grid-template-columns: repeat(2, minmax(0, 1fr));  
  }  
}  
  
@media (max-width: 420px) {  
  .page {  
    padding: 12px 10px 28px;  
  }  
  
  .header-panel {  
    top: 8px;  
    margin-bottom: 14px;  
    padding: 14px;  
    border-radius: 24px;  
  }  
  
  .nav-row {  
    margin-bottom: 14px;  
  }  
  
  .back-btn,  
  .header-badge {  
    padding: 9px 10px;  
  }  
  
  .title {  
    font-size: 26px;  
    line-height: 1.2;  
  }  
  
  .subtitle {  
    margin-top: 6px;  
    line-height: 1.55;  
  }  
  
  .progress-panel {  
    padding: 14px;  
  }  
  
  .section-header {  
    padding: 2px 2px 0;
```

```
    }  
  }  
</style>
```

```
// File: src/pages/Game.vue  
<script setup>  
import { computed, onMounted, onUnmounted, ref, watch } from 'vue'  
import { useRouter } from 'vue-router'  
import { AlertCircle, ArrowLeft, CheckCircle2, RotateCcw, Star } from 'lucide-vue-next'  
import { getDifficultyById } from '../config/difficulty'  
import { GAME_CONFIG, STORAGE_KEYS } from '../config/constants'  
import { useGame } from '../composables/useGame'  
import { useStorage } from '../composables/useStorage'  
import { useToast } from '../composables/useToast'  
import { useSound } from '../composables/useSound'  
import { getStarCount } from '../utils/stars'  
import NumberPad from '../components/NumberPad.vue'  
import QuestionCard from '../components/QuestionCard.vue'  
import ResultModal from '../components/ResultModal.vue'  
import ScoreBoard from '../components/ScoreBoard.vue'  
  
const props = defineProps({  
  id: {  
    type: String,  
    required: true  
  }  
})  
  
const router = useRouter()  
const difficulty = getDifficultyById(parseInt(props.id))  
  
if (!difficulty) {  
  router.replace('/difficulty')  
}  
  
const { getLeaderboard, updateBestScore } = useStorage()  
const { error: showError } = useToast()  
const {  
  playCorrect,  
  playWrong,  
  playVictory,  
  playUnlock,  
  playResultPraise,  
  playBack,  
  playKeyPress,  
  playDelete,  
  playSubmit,  
  stopPraise  
} = useSound()
```

```
const game = useGame(difficulty)
const showAnswer = ref(false)
const isWaiting = ref(false)
const userAnswer = ref('')
const modalResult = ref(null)
const modalVisible = ref(false)
const isLoading = ref(true)
const currentFeedbackState = ref('idle')
const streakCount = ref(0)
const showStreakReward = ref(false)
const streakRewardText = ref('')
const showNumberBondHint = ref(false)

watch(modalResult, (result, prev) => {
  if (result && !prev) {
    modalVisible.value = true
  } else if (!result && prev) {
    modalVisible.value = false
  }
})

const lastInputTime = ref(0)
const lastSubmitTime = ref(0)
const INPUT_DEBOUNCE = 45
const SUBMIT_DEBOUNCE = 120

let feedbackTimeout = null
let streakRewardTimeout = null
let unlockTimeout = null
let resultPraiseTimeout = null
let retryQuestions = null
const isReviewRound = ref(false)

const isComplete = computed(() => game.isComplete.value)
const currentQuestion = computed(() => game.currentQuestion.value)
const isCorrect = computed(() => currentQuestion.value?.isCorrect === true)
const isIncorrect = computed(() => currentQuestion.value?.isCorrect === false)
const shouldShowFeedback = computed(() => showAnswer.value && currentQuestion.value?.userAnswer !== null)
const currentQuestionHasNumberBondHint = computed(() => {
  const question = currentQuestion.value
  if (!question) return false

  const missingPart = question.missingPart || 'answer'
  let ballCount = 0

  if (question.operator === '+' && missingPart !== 'answer') {
    ballCount = question.result || question.answer || 0
  } else if (question.operator === '+') {
    ballCount = (Number(question.operand1) || 0) + (Number(question.operand2) || 0)
  } else if (question.operator === '-') {
```

```
    ballCount = Number(question.operand1) || 0
  }

  return ballCount > 0
})

function loadNumberBondHintSetting() {
  try {
    const saved = localStorage.getItem(STORAGE_KEYS.NUMBER_BOND_HINT)
    showNumberBondHint.value = saved === null ? false : saved === 'true'
  } catch (error) {
    console.error('读取小球提示设置失败:', error)
    showNumberBondHint.value = false
  }
}

function toggleNumberBondHint() {
  showNumberBondHint.value = !showNumberBondHint.value

  try {
    localStorage.setItem(STORAGE_KEYS.NUMBER_BOND_HINT, String(showNumberBondHint.value))
  } catch (error) {
    console.error('保存小球提示设置失败:', error)
  }

  playKeyPress()
}

function triggerHapticFeedback(level = 'medium') {
  if (navigator.vibrate) {
    if (level === 'light') {
      navigator.vibrate(20)
      return
    }

    if (level === 'strong') {
      navigator.vibrate([28, 20, 32])
      return
    }

    if (level === 'error') {
      navigator.vibrate([24, 24, 24])
      return
    }

    navigator.vibrate(50)
  }
}

function initGame() {
```

```
isLoading.value = true
game.startGame(retryQuestions ? { questions: retryQuestions } : undefined)
retryQuestions = null
isLoading.value = false
}

function submitAnswer() {
  if (isWaiting.value || !userAnswer.value || userAnswer.value.trim() === '') {
    showError('先写数字')
    return
  }

  const now = Date.now()
  if (now - lastSubmitTime.value < SUBMIT_DEBOUNCE) return
  lastSubmitTime.value = now

  const answer = parseInt(userAnswer.value)
  const correct = game.submitAnswer(answer)

  showAnswer.value = true
  isWaiting.value = true
  playSubmit()

  if (correct) {
    streakCount.value += 1
    triggerStreakReward()
    triggerHapticFeedback('strong')
    playCorrect()
  } else {
    streakCount.value = 0
    showStreakReward.value = false
    triggerHapticFeedback('error')
    playWrong()
  }

  if (correct) {
    currentFeedbackState.value = 'correct-auto'
    feedbackTimeout = setTimeout(() => {
      handleNextQuestion()
    }, GAME_CONFIG.FEEDBACK_DELAY)
    return
  }

  currentFeedbackState.value = 'incorrect-manual'
}

function triggerStreakReward() {
  const isFirstReward = streakCount.value === 3
  const isMilestoneReward = streakCount.value > 0 && streakCount.value % 5 === 0
```

```
    if (!isFirstReward && !isMilestoneReward) {
      return
    }

    streakRewardText.value = `连对${streakCount.value}题`
    showStreakReward.value = true

    if (streakRewardTimeout) {
      clearTimeout(streakRewardTimeout)
    }

    streakRewardTimeout = setTimeout(() => {
      showStreakReward.value = false
    }, 760)
  }

function handleInput(num) {
  if (isWaiting.value) return

  const now = Date.now()
  if (now - lastInputTime.value < INPUT_DEBOUNCE) return
  lastInputTime.value = now

  if (userAnswer.value.length < GAME_CONFIG.MAX_ANSWER_LENGTH) {
    userAnswer.value += num
    playKeyPress()
    return
  }

  triggerHapticFeedback('light')
}

function handleDelete() {
  if (isWaiting.value) return

  const now = Date.now()
  if (now - lastInputTime.value < INPUT_DEBOUNCE) return
  lastInputTime.value = now
  userAnswer.value = userAnswer.value.slice(0, -1)
  playDelete()
}

function handleNextQuestion() {
  if (game.currentIndex.value >= game.questions.value.length - 1) {
    showAnswer.value = false
    isWaiting.value = false
    handleGameComplete()
    return
  }
}
```

```
userAnswer.value = ''
showAnswer.value = false
isWaiting.value = false
currentFeedbackState.value = 'idle'
game.nextQuestion()
}

function handleFeedbackClick() {
  if (!isCorrect.value && currentFeedbackState.value === 'incorrect-manual') {
    currentFeedbackState.value = 'idle'
    handleNextQuestion()
  }
}

function handleGameComplete() {
  game.completeGame()
  const result = {
    ...game.getResult(),
    isReviewRound: isReviewRound.value
  }
  const updateResult = result.isReviewRound
    ? {
      isNewBest: false,
      leaderboard: getLeaderboard(parseInt(props.id, 10), result.totalCount),
      leaderboardRank: null
    }
    : updateBestScore(parseInt(props.id, 10), result)
  const stars = getStarCount(result.accuracy)
  const didPass = result.accuracy >= GAME_CONFIG.PASS_ACCURACY

  triggerHapticFeedback(stars >= 4 ? 'strong' : 'medium')

  modalResult.value = {
    ...result,
    isNewBest: updateResult.isNewBest,
    leaderboard: updateResult.leaderboard || [],
    leaderboardRank: updateResult.leaderboardRank
  }

  if (didPass) {
    playVictory()

    if (updateResult.isNewBest) {
      unlockTimeout = setTimeout(() => playUnlock(), 460)
    }
  }

  resultPraiseTimeout = setTimeout(() => {
    playResultPraise({
      accuracy: result.accuracy,
```



```
        isNewBest: updateResult.isNewBest,
        isReviewRound: result.isReviewRound
    })
}, didPass ? (updateResult.isNewBest ? 920 : 520) : 320)
}

function goBack() {
    playBack()
    router.push('/difficulty')
}

function resetRound({ preserveRetryQuestions = false, reviewRound = false } = {}) {
    stopPraise()

    if (!preserveRetryQuestions) {
        retryQuestions = null
    }

    isReviewRound.value = reviewRound
    modalResult.value = null
    userAnswer.value = ''
    showAnswer.value = false
    isWaiting.value = false
    currentFeedbackState.value = 'idle'
    streakCount.value = 0
    showStreakReward.value = false

    if (feedbackTimeout) {
        clearTimeout(feedbackTimeout)
        feedbackTimeout = null
    }

    if (streakRewardTimeout) {
        clearTimeout(streakRewardTimeout)
        streakRewardTimeout = null
    }

    if (unlockTimeout) {
        clearTimeout(unlockTimeout)
        unlockTimeout = null
    }

    if (resultPraiseTimeout) {
        clearTimeout(resultPraiseTimeout)
        resultPraiseTimeout = null
    }

    initGame()
}
```

```
function handleRetry() {
  resetRound()
}

function handleRetryMistakes() {
  const incorrectQuestions = modalResult.value?.incorrectQuestions || []
  if (!incorrectQuestions.length) {
    handleRetry()
    return
  }

  retryQuestions = incorrectQuestions.map((item) => ({
    operand1: item.operand1,
    operand2: item.operand2,
    operator: item.operator,
    result: item.result,
    missingPart: item.missingPart,
    answer: item.correctAnswer
  })))

  resetRound({
    preserveRetryQuestions: true,
    reviewRound: true
  })
}

function handleHome() {
  stopPraise()
  modalVisible.value = false
  setTimeout(() => {
    modalResult.value = null
    router.push('/difficulty')
  }, 300)
}

function handleKeyPress(event) {
  if (event.key >= '0' && event.key <= '9') {
    handleInput(parseInt(event.key))
  } else if (event.key === 'Backspace') {
    handleDelete()
  } else if (event.key === 'Enter') {
    submitAnswer()
  }
}

onMounted(() => {
  window.scrollTo(0, 0)
  loadNumberBondHintSetting()
  initGame()
  window.addEventListener('keydown', handleKeyPress)
```

```
  })

  onUnmounted(() => {
    window.removeEventListener('keydown', handleKeyPress)
    stopPraise()

    if (feedbackTimeout) {
      clearTimeout(feedbackTimeout)
      feedbackTimeout = null
    }

    if (streakRewardTimeout) {
      clearTimeout(streakRewardTimeout)
      streakRewardTimeout = null
    }

    if (unlockTimeout) {
      clearTimeout(unlockTimeout)
      unlockTimeout = null
    }

    if (resultPraiseTimeout) {
      clearTimeout(resultPraiseTimeout)
      resultPraiseTimeout = null
    }
  })
</script>

<template>
  <div v-if="difficulty" class="page">
    <Transition name="fade">
      <div v-if="isLoading" class="loading-overlay">
        <div class="loading-panel">
          <div class="spinner"></div>
          <p class="loading-text text-child-base">准备中</p>
        </div>
      </div>
    </Transition>

    <header class="header-panel">
      <button class="nav-btn" @click="goBack" aria-label="返回关卡页">
        <ArrowLeft :size="20" />
      </button>

      <div class="title-group">
        <p class="eyebrow">来做题</p>
        <h1 class="title text-child-lg">第{{ difficulty.id }}关</h1>
        <p class="subtitle text-child-sm">{{ difficulty.description }}</p>
      </div>
    </header>
  </div>
</template>
```

```
<button class="nav-btn nav-btn-accent" title="重新开始" aria-label="重新开始" @click="handleRetry">
  <RotateCcw :size="18" />
</button>
</header>

<main class="main-layout">
  <section class="question-section">
    <Transition name="streak-reward">
      <div v-if="showStreakReward" class="streak-reward">
        <Star :size="16" fill="currentColor" />
        <span>{{ streakRewardText }}</span>
      </div>
    </Transition>

    <div
      v-if="game.currentQuestion.value"
      class="question-shell"
    >
      <QuestionCard
        :class="{ 'is-hidden-on-error': isIncorrect && shouldShowFeedback }"
        :question="game.currentQuestion.value"
        :difficulty="difficulty"
        :show-answer="showAnswer"
        :user-answer="userAnswer"
        :current-index="game.currentIndex.value"
        :total-questions="game.questions.value.length"
        :show-number-bond-hint="showNumberBondHint"
        :show-number-bond-hint-toggle="currentQuestionHasNumberBondHint"
        @toggle-number-bond-hint="toggleNumberBondHint"
      />
    </div>

    <Transition name="feedback">
      <div
        v-if="shouldShowFeedback"
        class="feedback-wrap"
        :class="{ 'is-success': isCorrect, 'is-error': isIncorrect }"
        @click="handleFeedbackClick"
      >
        <!-- 答错时的对比模态内容 -->
        <template v-if="isIncorrect">
          <div class="error-review-container">
            <!-- 这里的 QuestionCard 会显示用户的错误答案 -->
            <QuestionCard
              :question="game.currentQuestion.value"
              :difficulty="difficulty"
              :show-answer="true"
              :user-answer="userAnswer"
              :current-index="game.currentIndex.value"
              :total-questions="game.questions.value.length"
            </div>
          </template>
        </div>
      </Transition>
    </div>
  </section>
</main>
```

```
      :show-number-bond-hint="showNumberBondHint"
      class="review-question"
    />

    <div class="feedback-card error">
      <div class="feedback-icon error">
        <AlertCircle :size="22" />
      </div>
      <p class="feedback-kicker">正确答案是</p>
      <strong class="feedback-main">{{ currentQuestion.answer }}</strong>
      <p class="feedback-tap-note">看清楚后，点一下继续</p>
    </div>
  </div>
</template>

<!-- 答对时的轻量反馈 -->
<template v-else>
  <div class="feedback-card success">
    <div class="feedback-icon success">
      <CheckCircle2 :size="22" />
    </div>
    <strong class="feedback-main">对啦</strong>
  </div>
</template>
</div>
</Transition>
</section>

<section class="control-section">
  <div class="score-wrap">
    <ScoreBoard
      :current-index="game.currentIndex.value"
      :total-questions="game.questions.value.length"
      :correct-count="game.correctCount.value"
      :streak="streakCount"
    />
  </div>

  <div class="keypad-wrap">
    <NumberPad
      :disabled="isWaiting || isComplete"
      @input="handleInput"
      @delete="handleDelete"
      @submit="submitAnswer"
    />
  </div>

</section>
</main>
```

```
<ResultModal
  v-if="modalResult"
  :show="modalVisible"
  :result="modalResult"
  :is-new-best="modalResult.isNewBest"
  :difficulty-id="props.id"
  :leaderboard="modalResult.leaderboard"
  :leaderboard-rank="modalResult.leaderboardRank"
  @retry="handleRetry"
  @retry-mistakes="handleRetryMistakes"
  @home="handleHome"
/>
</div>
</template>

<style scoped>
.page {
  min-height: 100dvh;
  padding: calc(env(safe-area-inset-top, 0px) + 12px) 12px 12px;
}

.header-panel,
.loading-panel {
  background: rgba(255, 255, 255, 0.94);
  border: 1px solid var(--border-light);
  box-shadow: var(--shadow-sm);
}

.header-panel {
  display: grid;
  grid-template-columns: 48px 1fr 48px;
  align-items: center;
  gap: 12px;
  margin-bottom: 12px;
  padding: 12px;
  border-radius: var(--radius-xl);
}

.nav-btn,
.loading-panel {
  display: flex;
  align-items: center;
  justify-content: center;
}

.nav-btn {
  width: 48px;
  height: 48px;
  border: none;
  border-radius: var(--radius-sm);
```

```
background: rgba(255, 255, 255, 0.92);
color: var(--text-secondary);
}

.nav-btn:active {
  transform: scale(0.97);
}

@media (hover: hover) {
  .nav-btn:hover {
    color: var(--text-primary);
    background: rgba(255, 255, 255, 0.92);
  }
}

.nav-btn-accent {
  color: var(--candy-blue-dark);
  background: rgba(49, 120, 246, 0.08);
}

.title-group {
  min-width: 0;
  text-align: center;
}

.eyebrow {
  margin-bottom: 4px;
  color: var(--candy-blue-dark);
  font-size: 12px;
  font-weight: 800;
  letter-spacing: 0.1em;
  text-transform: uppercase;
}

.title {
  color: var(--text-primary);
  font-weight: 800;
}

.subtitle {
  margin-top: 2px;
  color: var(--text-secondary);
  line-height: 1.35;
}

.main-layout {
  display: grid;
  gap: 10px;
}
```

```
.question-section,
.control-section {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.question-section {
  position: relative;
  justify-content: flex-start;
  min-height: 142px;
}

.question-shell {
  position: relative;
  width: min(100%, 640px);
  margin: 0 auto;
}

.is-hidden-on-error {
  opacity: 0;
  visibility: hidden;
  pointer-events: none;
}

.score-wrap {
  order: 1;
}

.keypad-wrap {
  order: 2;
}

.feedback-wrap {
  position: absolute;
  inset: 0;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 12px;
  pointer-events: none;
  border-radius: var(--radius-lg);
  z-index: 5;
}

.feedback-wrap.is-success {
  align-items: flex-start;
  padding-top: 12px;
}
```



```
.feedback-wrap.is-error {
  position: fixed; /* 全屏拦截 */
  inset: 0;
  z-index: 100;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 16px;
  pointer-events: auto;
  background: rgba(18, 30, 49, 0.45);
  backdrop-filter: blur(6px);
}

.error-review-container {
  width: 100%;
  max-width: 440px;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 16px; /* 题目和答案之间的间距 */
  animation: feedbackSlideUp 0.4s var(--ease-out);
}

.review-question {
  width: 100% !important;
  background: white !important;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.2) !important;
  transform: scale(1.02);
}

.feedback-card {
  width: 100%;
  max-width: 360px;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 8px;
  padding: 20px 18px 18px;
  border-radius: var(--radius-lg);
  background: rgba(255, 255, 255, 0.98);
  border: 1px solid var(--border-default);
  box-shadow: var(--shadow-sm);
}

.feedback-card.success {
  position: absolute;
  top: 12px;
  width: min(100%, 340px);
  flex-direction: row;
  justify-content: center;
```

```
gap: 10px;
padding: 14px 16px;
background: rgba(46, 196, 182, 0.1);
border: 2px solid rgba(78, 205, 196, 0.28);
box-shadow: var(--shadow-sm);
pointer-events: none;
}

.feedback-card.error {
  background: white;
  border: 4px solid var(--candy-red-dark);
  box-shadow: 0 12px 32px rgba(255, 107, 107, 0.3);
  pointer-events: auto;
  cursor: pointer;
}

@keyframes feedbackSlideUp {
  from { transform: translateY(20px); opacity: 0; }
  to { transform: translateY(0); opacity: 1; }
}

.feedback-icon {
  width: 36px;
  height: 36px;
  border-radius: var(--radius-xs);
  display: flex;
  align-items: center;
  justify-content: center;
}

.feedback-icon.success {
  color: var(--candy-mint-dark);
  background: rgba(78, 205, 196, 0.18);
}

.feedback-icon.error {
  color: var(--candy-red-dark);
  background: rgba(255, 107, 107, 0.12);
}

.feedback-text {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 2px;
}

.feedback-kicker {
  color: var(--text-primary);
  font-size: var(--font-base);
```

```
    font-weight: 800;
  }

.feedback-main {
  color: #2D2D2D;
  font-size: 32px;
  font-weight: 800;
  line-height: 1.1;
}

.feedback-card.success .feedback-main {
  font-size: 20px;
  color: var(--candy-mint-dark);
}

.feedback-card.error .feedback-main {
  color: var(--candy-red-dark);
  font-size: 52px;
  line-height: 1;
}

.feedback-tap-note {
  color: var(--text-secondary);
  font-size: var(--font-sm);
  font-weight: 700;
}

.loading-overlay {
  position: fixed;
  inset: 0;
  z-index: 40;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 20px;
  background: rgba(238, 244, 255, 0.84);
}

.loading-panel {
  flex-direction: column;
  gap: 14px;
  width: 180px;
  height: 160px;
  border-radius: var(--radius-xl);
}

.spinner {
  width: 42px;
  height: 42px;
  border: 4px solid rgba(49, 120, 246, 0.14);
```

```
border-top-color: var(--candy-blue);
border-radius: 50%;
animation: spin 0.8s linear infinite;
}

.loading-text {
  color: var(--text-secondary);
  font-weight: 700;
}

.feedback-enter-active,
.feedback-leave-active {
  transition: opacity var(--duration-normal) var(--ease-out), transform var(--duration-normal) var(--ease-out);
}

.feedback-enter-from,
.feedback-leave-to {
  opacity: 0;
  transform: translateY(16px);
}

@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}

@media (min-width: 960px) {
  .page {
    max-width: 1120px;
    margin: 0 auto;
    padding: 20px;
  }

  .main-layout {
    grid-template-columns: minmax(0, 1.2fr) minmax(320px, 360px);
    align-items: start;
  }

  .question-section {
    justify-content: center;
    min-height: 500px;
  }

  .control-section {
    position: sticky;
    top: 20px;
  }
}
```

```
@media (max-width: 420px) {  
  .page {  
    padding: 8px;  
  }  
  
  .header-panel {  
    grid-template-columns: 44px 1fr 44px;  
    gap: 8px;  
    margin-bottom: 10px;  
    padding: 10px;  
    border-radius: var(--radius-lg);  
  }  
  
  .nav-btn {  
    width: 44px;  
    height: 44px;  
    border-radius: var(--radius-sm);  
  }  
  
  .title {  
    font-size: 18px;  
    line-height: 1.2;  
  }  
  
  .eyebrow {  
    margin-bottom: 2px;  
    font-size: 11px;  
  }  
  
  .subtitle {  
    margin-top: 2px;  
    font-size: 12px;  
    line-height: 1.45;  
  }  
  
  .main-layout {  
    gap: 8px;  
  }  
  
  .question-section,  
  .control-section {  
    gap: 8px;  
  }  
  
  .question-section {  
    min-height: 124px;  
  }  
  
  .feedback-card {  
    padding: 16px 14px;  
  }  
}
```

```
    border-radius: var(--radius-lg);
  }

  .feedback-card.success {
    padding: 12px 14px;
    border-radius: var(--radius-sm);
  }

  .streak-reward {
    top: 4px;
    right: 4px;
    padding: 6px 9px;
    font-size: 11px;
  }

  .feedback-main {
    font-size: 28px;
  }

  .feedback-card.success .feedback-main {
    font-size: 17px;
  }

  .feedback-card.error .feedback-main {
    font-size: 44px;
  }
}

@media (max-width: 959px) and (max-height: 860px) {
  .header-panel {
    grid-template-columns: 42px 1fr 42px;
    gap: 8px;
    margin-bottom: 8px;
    padding: 8px;
  }

  .nav-btn {
    width: 42px;
    height: 42px;
    border-radius: var(--radius-xs);
  }

  .eyebrow,
  .subtitle {
    display: none;
  }

  .title {
    font-size: 18px;
    line-height: 1.15;
  }
}
```

```

    if (btn) {
      btn.classList.add('is-leaving')
    }
    setTimeout(() => {
      router.push(route)
    }, NAVIGATION_DELAY)
  }

function startGame(event) {
  navigateTo('/difficulty', event)
}

function goToExplore(event) {
  navigateTo('/explore', event)
}

onMounted(() => {
  requestAnimationFrame(() => {
    isReady.value = true
  })
})
</script>

<template>
  <div class="home-page">
    <!-- 动态背景 -->
    <div class="bg-decorations">
      <span
        v-for="(sym, i) in floatingSymbols"
        :key="i"
        class="float-item"
        :style="{ '--delay: `${i * 1.5}s`, '--left: `${10 + i * 15}%`, '--top: `${20 + (i % 3) * 20}%` }"
      >
        {{ sym }}
      </span>
    </div>

    <div class="home-shell" :class="{ 'is-ready': isReady }">
      <header class="logo-area">
        <div class="logo-circle">
          <Sparkles :size="32" class="logo-sparkle" />
          <div class="logo-inner">
            <span class="logo-plus">+</span>
            <span class="logo-minus">-</span>
          </div>
        </div>
      </header>

      <section class="hero-panel">
        <div class="hero-topline">

```

```
<span class="mission-chip">
  <Sparkles :size="14" />
  <span>启蒙数学</span>
</span>
<span class="mission-note">{{ TOTAL_LEVELS }}关卡</span>
</div>

<div class="hero-copy">
  <h1 class="title">算一算</h1>
  <p class="subtitle">有趣又好玩的数学游戏。</p>
</div>

<div v-if="completedCount > 0 || (stats.totalCorrect || 0) > 0" class="stats-grid">
  <div class="stat-item">
    <span class="stat-value">{{ completedCount }}</span>
    <span class="stat-label">已通关</span>
  </div>
  <div class="stat-divider"></div>
  <div class="stat-item">
    <span class="stat-value">{{ stats.totalCorrect || 0 }}</span>
    <span class="stat-label">累计答对</span>
  </div>
</div>

<div v-else class="welcome-hint">
  <Sparkles :size="20" class="welcome-sparkle" />
  <span>开始你的第一关吧! </span>
</div>
</section>

<section class="action-panel">
  <button class="btn-main" data-testid="start-challenge-btn" @click="startGame($event)">
    <span class="btn-main-icon">
      <Play :size="24" />
    </span>
    <span class="btn-text">开始挑战</span>
    <ArrowRight :size="20" class="btn-arrow" />
  </button>
  <button class="btn-main btn-explore" data-testid="explore-btn" @click="goToExplore($event)">
    <span class="btn-main-icon">
      <Binary :size="24" />
    </span>
    <span class="btn-text">数字探索</span>
    <ArrowRight :size="20" class="btn-arrow" />
  </button>
</section>
</div>
</div>
</template>

<style scoped>
```



```
.home-page {  
  min-height: 100vh;  
  min-height: 100dvh;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  padding: 24px 18px;  
  background: radial-gradient(circle at top left, #f8faff 0%, #eef4ff 100%);  
  position: relative;  
  overflow: hidden;  
}
```

```
/* 动态背景装饰 */
```

```
.bg-decorations {  
  position: absolute;  
  inset: 0;  
  pointer-events: none;  
  z-index: 0;  
}
```

```
.float-item {  
  position: absolute;  
  left: var(--left);  
  top: var(--top);  
  font-size: 28px;  
  font-weight: 900;  
  color: rgba(49, 120, 246, 0.04);  
  animation: float 6s ease-in-out infinite;  
  animation-delay: var(--delay);  
  user-select: none;  
}
```

```
@keyframes float {  
  0%, 100% { transform: translateY(0) rotate(0); }  
  50% { transform: translateY(-20px) rotate(10deg); }  
}
```

```
.home-shell {  
  width: 100%;  
  max-width: 440px;  
  display: flex;  
  flex-direction: column;  
  gap: 20px;  
  position: relative;  
  z-index: 1;  
}
```

```
/* Logo 区域 */
```

```
.logo-area {  
  display: flex;
```

```
    justify-content: center;
    margin-bottom: 8px;
    opacity: 0;
    transform: scale(0.9);
    transition: all 0.6s var(--ease-out);
}

.is-ready .logo-area {
    opacity: 1;
    transform: scale(1);
}

.logo-circle {
    width: 84px;
    height: 84px;
    border-radius: 30px;
    background: white;
    display: flex;
    align-items: center;
    justify-content: center;
    position: relative;
    box-shadow: 0 8px 32px rgba(49, 120, 246, 0.12), inset 0 2px 4px white;
    border: 1px solid rgba(255, 255, 255, 0.8);
}

.logo-inner {
    display: flex;
    gap: 4px;
    font-size: 36px;
    font-weight: 900;
    line-height: 1;
}

.logo-plus { color: var(--candy-blue-dark); }
.logo-minus { color: var(--candy-mint-dark); }

.logo-sparkle {
    position: absolute;
    top: -12px;
    right: -12px;
    color: var(--candy-yellow-dark);
    filter: drop-shadow(0 0 8px rgba(245, 201, 74, 0.4));
    animation: pulse 2s ease-in-out infinite;
}

@keyframes pulse {
    0%, 100% { transform: scale(1); opacity: 1; }
    50% { transform: scale(1.1); opacity: 0.8; }
}
```

```
/* 面板通用样式 */
.hero-panel,
.action-panel {
  background: rgba(255, 255, 255, 0.72);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  border: 1px solid rgba(255, 255, 255, 0.8);
  box-shadow: 0 12px 40px rgba(0, 0, 0, 0.04);
  opacity: 0;
  transform: translateY(16px);
  transition: opacity 0.6s var(--ease-out), transform 0.6s var(--ease-out);
}

.is-ready .hero-panel { transition-delay: 150ms; opacity: 1; transform: translateY(0); }
.is-ready .action-panel { transition-delay: 250ms; opacity: 1; transform: translateY(0); }

.hero-panel {
  border-radius: 32px;
  padding: 24px;
  text-align: center;
}

.hero-topline {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 12px;
  margin-bottom: 24px;
}

.mission-chip {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  padding: 6px 12px;
  border-radius: 999px;
  background: rgba(49, 120, 246, 0.06);
  color: var(--text-blue);
  font-size: 13px;
  font-weight: 700;
}

.mission-note {
  color: var(--text-secondary);
  font-size: 13px;
  font-weight: 600;
}

.hero-copy {
  margin-bottom: 28px;
}
```

```
}

.title {
  margin-bottom: 8px;
  color: var(--text-primary);
  font-size: 42px;
  font-weight: 800;
  letter-spacing: -0.02em;
}

.subtitle {
  color: var(--text-secondary);
  font-size: 16px;
  font-weight: 500;
}

/* 统计网格 */
.stats-grid {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 20px;
  padding: 16px;
  background: rgba(255, 255, 255, 0.5);
  border-radius: 20px;
  border: 1px solid rgba(255, 255, 255, 0.6);
}

.stat-item {
  flex: 1;
  display: flex;
  flex-direction: column;
  gap: 2px;
}

.stat-value {
  font-size: 24px;
  font-weight: 800;
  color: var(--text-primary);
  font-variant-numeric: tabular-nums;
}

.stat-label {
  font-size: 12px;
  font-weight: 600;
  color: var(--text-secondary);
}

.stat-divider {
  width: 1px;
```

```
    height: 32px;
    background: rgba(0, 0, 0, 0.06);
}
```

```
/* 首次用户欢迎提示 */
```

```
.welcome-hint {
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 8px;
    padding: 16px;
    background: rgba(255, 255, 255, 0.5);
    border-radius: 20px;
    border: 1px solid rgba(255, 255, 255, 0.6);
    color: var(--text-secondary);
    font-size: 14px;
    font-weight: 600;
}
```

```
.welcome-sparkle {
    color: var(--candy-yellow-dark);
    animation: pulse 2s ease-in-out infinite;
}
```

```
/* 操作区域 */
```

```
.action-panel {
    border-radius: 28px;
    padding: 12px;
    display: flex;
    flex-direction: column;
    gap: 12px;
}
```

```
.btn-main {
    width: 100%;
    height: 64px;
    border: none;
    border-radius: 22px;
    background: var(--candy-blue-dark);
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 16px;
    font-size: 18px;
    font-weight: 800;
    box-shadow: 0 8px 24px rgba(49, 120, 246, 0.2);
    transition: all 0.3s var(--ease-out);
    cursor: pointer;
}
```

```
.btn-main-icon {
  width: 32px;
  height: 32px;
  background: rgba(255, 255, 255, 0.2);
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
}

.btn-text {
  letter-spacing: 0.02em;
}

.btn-arrow {
  transition: transform 0.3s var(--ease-out);
}

.btn-main:active {
  transform: scale(0.96);
  background: var(--brand-primary-dark);
}

.btn-main.is-leaving {
  opacity: 0.5;
  transform: scale(0.9) translateY(4px);
  filter: blur(2px);
}

@media (hover: hover) {
  .btn-main:hover {
    transform: translateY(-2px);
    box-shadow: 0 12px 32px rgba(49, 120, 246, 0.3);
  }

  .btn-main:hover .btn-arrow {
    transform: translateX(4px);
  }
}

@media (max-width: 420px) {
  .home-shell { gap: 16px; }
  .title { font-size: 36px; }
  .hero-panel { padding: 20px; }
  .stats-grid { gap: 12px; padding: 12px; }
  .stat-value { font-size: 20px; }
  .logo-circle { width: 72px; height: 72px; border-radius: 24px; }
  .logo-inner { font-size: 30px; }
}
```

```
</style>
```

```
// File: src/pages/NumberExplore.vue
```

```
<script setup>
```

```
import { computed, ref } from 'vue'
```

```
import { useRouter } from 'vue-router'
```

```
import { ArrowLeft, RotateCow, Target } from 'lucide-vue-next'
```

```
import NumberPad from '../components/NumberPad.vue'
```

```
import BallArray from '../components/BallArray.vue'
```

```
import { useSound } from '../composables/useSound'
```

```
const router = useRouter()
```

```
const { playClick, playSubmit } = useSound()
```

```
const QUICK_COUNTS = [10, 50, 100, 500, 1000]
```

```
const CHALLENGE_RANGES = [
```

```
  { label: '10 以内', min: 1, max: 10, hint: '先从少量小球开始，培养数量感。' },
  { label: '10 几', min: 11, max: 19, hint: '开始感受“十几”这个区间。' },
  { label: '20 几', min: 20, max: 29, hint: '先练习 20 多的数量，观察两排多一点的感觉。' },
  { label: '30 几', min: 30, max: 39, hint: '继续往上，看看 30 多和 20 多的差别。' },
  { label: '40 几', min: 40, max: 49, hint: '球阵会更满一些，试着先估整排。' },
  { label: '50 几', min: 50, max: 59, hint: '开始建立“半百左右”的数量感。' },
  { label: '60 几', min: 60, max: 69, hint: '先看大概几排，再判断剩下多少。' },
  { label: '70 几', min: 70, max: 79, hint: '数量越来越密了，先估整十会更稳。' },
  { label: '80 几', min: 80, max: 89, hint: '快接近 100 了，看看视觉上有有多满。' },
  { label: '90 几', min: 90, max: 99, hint: '最后冲刺到 100 前，练习接近满格的感觉。' },
  { label: '100-120', min: 100, max: 120, hint: '刚过 100，先感受一层和两层之间的变化。' },
  { label: '121-150', min: 121, max: 150, hint: '继续练习 100 多，开始观察每层的大致分布。' },
  { label: '151-199', min: 151, max: 199, hint: '接近 200 之前，试着同时看层数和单层数量。' },
  { label: '200-300', min: 200, max: 300, hint: '进入几百个范围后，先估层数会更容易。' },
  { label: '301-500', min: 301, max: 500, hint: '球阵更厚了，适合练“整百 + 零头”的感觉。' },
  { label: '501-750', min: 501, max: 750, hint: '开始挑战大数量，先看大体体积再细估。' },
  { label: '751-1000', min: 751, max: 1000, hint: '最终高阶范围，练习接近满载的数量判断。' }
]
```

```
const mode = ref('explore')
```

```
const inputNumber = ref('')
```

```
const currentCount = ref(0)
```

```
const showResult = ref(false)
```

```
const isShaking = ref(false)
```

```
const statusMessage = ref('输入 1-1000，看看它会变成多少颗小球。')
```

```
const statusTone = ref('default')
```

```
const challengeTargetCount = ref(0)
```

```
const challengeRangeIndex = ref(0)
```

```
const challengeCorrectInRange = ref(0)
```

```
const challengeSolvedCount = ref(0)
```

```
const challengeLastGuess = ref(null)
```

```
const challengeLastCorrect = ref(false)
```

```

const isChallengeMode = computed(() => mode.value === 'challenge')
const currentChallengeRange = computed(() => CHALLENGE_RANGES[challengeRangeIndex.value])
const challengeProgressText = computed(() => `本档答对 ${challengeCorrectInRange.value} 题`)
const challengeStatusText = computed(() => {
  if (!isChallengeMode.value) return ""
  return `当前范围: ${currentChallengeRange.value.label} · ${challengeProgressText.value}`
})

const numberParts = computed(() => {
  const count = currentCount.value
  const hundreds = Math.floor(count / 100)
  const tens = Math.floor((count % 100) / 10)
  const ones = count % 10
  const parts = []

  if (hundreds > 0) parts.push(`${hundreds}个百`)
  if (tens > 0) parts.push(`${tens}个十`)
  if (ones > 0 || parts.length === 0) parts.push(`${ones}个一`)

  return parts.join(' + ')
})

function setStatus(message, tone = 'default') {
  statusMessage.value = message
  statusTone.value = tone
}

function pulseNumber() {
  isShaking.value = true
  window.setTimeout(() => {
    isShaking.value = false
  }, 280)
}

/**
 * Generates a random integer within a range.
 * @param {number} min – Minimum value.
 * @param {number} max – Maximum value.
 * @returns {number} Random integer.
 */
function randomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min
}

function prepareExploreMode() {
  inputNumber.value = ""
  currentCount.value = 0
  showResult.value = false
  challengeLastGuess.value = null
  setStatus('输入 1-1000, 看看它会变成多少颗小球。', 'default')
}

```



```
}

function prepareChallengeRound(keepInput = false) {
  const range = currentChallengeRange.value

  challengeTargetCount.value = randomInt(range.min, range.max)
  showResult.value = false
  currentCount.value = 0
  challengeLastGuess.value = null
  challengeLastCorrect.value = false

  if (!keepInput) {
    inputNumber.value = ''
  }

  setStatus(`挑战范围: ${range.label}。${range.hint}`, 'default')
}

function startChallenge(resetProgress = false) {
  if (resetProgress) {
    challengeRangeIndex.value = 0
    challengeCorrectInRange.value = 0
    challengeSolvedCount.value = 0
  }

  prepareChallengeRound()
}

function selectChallengeRange(index) {
  if (challengeRangeIndex.value === index && !showResult.value) return

  playClick()
  challengeRangeIndex.value = index
  challengeCorrectInRange.value = 0
  prepareChallengeRound()
}

function switchMode(nextMode) {
  if (mode.value === nextMode) return

  playClick()
  mode.value = nextMode
  isShaking.value = false

  if (nextMode === 'challenge') {
    startChallenge(true)
    return
  }

  prepareExploreMode()
}
```

```
}

function handleInput(num) {
  playClick()

  if (inputNumber.value === '' && num === 0) {
    setStatus('从 1 开始输入，0 不能单独展示。', 'warning')
    pulseNumber()
    return
  }

  const nextValue = `${inputNumber.value}${num}`
  const parsed = Number.parseInt(nextValue, 10)

  if (Number.isNaN(parsed) || parsed > 1000) {
    setStatus('最大可以输入到 1000。', 'warning')
    pulseNumber()
    return
  }

  inputNumber.value = nextValue

  if (isChallengeMode.value) {
    setStatus(`你猜的是 ${parsed}。${challengeStatusText.value}`, 'default')
    return
  }

  setStatus(`准备探索 ${parsed}。`, 'default')
}

function handleDelete() {
  playClick()
  inputNumber.value = inputNumber.value.slice(0, -1)

  if (isChallengeMode.value) {
    setStatus(inputNumber.value ? `你猜的是 ${inputNumber.value}。${challengeStatusText.value}` : `${challengeStatusText.value}, 先看球阵再猜数字。`, 'default')
    return
  }

  setStatus(inputNumber.value ? `当前输入 ${inputNumber.value}。` : `输入 1-1000, 看看它会变成多少颗小球。`, 'default')
}

function handleExploreSubmit(num) {
  playSubmit()
  currentCount.value = num
  showResult.value = true
  setStatus(`正在探索 ${num}。`, 'success')
}

function handleChallengeSubmit(num) {
```

```
const target = challengeTargetCount.value
const correct = num === target

challengeLastGuess.value = num
challengeLastCorrect.value = correct
currentCount.value = target
showResult.value = true

if (correct) {
  playSubmit()
  challengeSolvedCount.value += 1
  challengeCorrectInRange.value += 1
  setStatus(`猜对了。答案就是 ${target}。`, 'success')
  return
}

playClick()
setStatus(`这次没猜中，正确答案是 ${target}。`, 'warning')
}

function handleSubmit() {
  const num = Number.parseInt(inputNumber.value, 10)

  if (Number.isNaN(num) || num < 1 || num > 1000) {
    playClick()
    setStatus(`请输入 1 到 1000 之间的数字。`, 'warning')
    pulseNumber()
    return
  }

  if (isChallengeMode.value) {
    handleChallengeSubmit(num)
    return
  }

  handleExploreSubmit(num)
}

function applyQuickCount(num) {
  playClick()
  inputNumber.value = String(num)
  setStatus(`已选择 ${num}，点击确认就能看结果。`, 'default')
}

function updateCurrentCount(nextCount) {
  const safeCount = Math.min(1000, Math.max(1, nextCount))

  if (safeCount === currentCount.value) {
    setStatus(`已经到 ${safeCount} 了。`, 'warning')
    pulseNumber()
  }
}
```

```
        return
    }

    playClick()
    currentCount.value = safeCount
    inputNumber.value = String(safeCount)
    setStatus(`正在探索 ${safeCount}。`, 'success')
}

function stepCount(step) {
    updateCurrentCount(currentCount.value + step)
}

function jumpToCount(num) {
    updateCurrentCount(num)
}

function resetExplore() {
    playClick()

    if (isChallengeMode.value) {
        startChallenge(false)
        return
    }

    prepareExploreMode()
}

function nextChallenge() {
    playClick()
    prepareChallengeRound()
}

function retryChallengeRange() {
    playClick()
    challengeCorrectInRange.value = 0
    prepareChallengeRound()
}

function goHome() {
    playClick()
    router.push('/')
}
</script>

<template>
<div class="explore-page">
    <Transition name="fade" mode="out-in">
        <div v-if="!showResult" class="view-wrapper">
            <header class="top-bar">
```

```

<button class="back-btn" aria-label="返回首页" @click="goHome">
  <ArrowLeft :size="22" stroke-width="2.8" />
</button>
</header>

<main class="input-screen">
  <section class="mode-switch" aria-label="模式切换">
    <button
      class="mode-btn"
      :class="{ 'is-active': mode === 'explore' }"
      type="button"
      @click="switchMode('explore')"
    >
      自由探索
    </button>
    <button
      class="mode-btn"
      :class="{ 'is-active': mode === 'challenge' }"
      type="button"
      @click="switchMode('challenge')"
    >
      挑战模式
    </button>
  </section>

  <section v-if="isChallengeMode" class="range-selector-shell">
    <div class="range-selector-header">
      <div class="counter-badge range-title-badge">选择训练范围</div>
      <p class="range-selector-copy">先选范围，再看球阵猜数字。</p>
    </div>

    <div class="range-selector" aria-label="挑战范围选择">
      <button
        v-for="(range, index) in CHALLENGE_RANGES"
        :key="range.label"
        class="range-chip"
        :class="{ 'is-active': challengeRangeIndex === index }"
        type="button"
        @click="selectChallengeRange(index)"
      >
        {{ range.label }}
      </button>
    </div>
  </section>

  <section
    :class="isChallengeMode ? 'challenge-workspace' : 'explore-workspace'"
    :aria-label="isChallengeMode ? '挑战练习区' : '自由探索区'"
  >
    <section class="number-stage" :class="{ 'is-shaking': isShaking }">

```

```

<div class="number-card">
  <div class="number-card-top">
    <div class="counter-badge">{{ isChallengeMode ? '数字挑战' : '数字探索' }}</div>
    <div v-if="isChallengeMode" class="counter-badge range-badge">{{ currentChallengeRange.label }}</div>
  </div>
  <div class="number-display font-number">
    <div class="big-number" :class="{ 'is-empty': !inputNumber }">
      {{ inputNumber || '?' }}
    </div>
  </div>
  <p v-if="!isChallengeMode" class="helper-copy">
    输入一个数字，马上看看它会变成多少颗球。
  </p>
  <p class="status-copy" :class="{ 'is-': statusTone }" aria-live="polite">
    {{ isChallengeMode ? challengeStatusText : statusMessage }}
  </p>

  <div v-if="isChallengeMode" class="challenge-meta">
    <div class="challenge-card">
      <span class="challenge-label">范围</span>
      <strong>{{ currentChallengeRange.label }}</strong>
    </div>
    <div class="challenge-card">
      <span class="challenge-label">本档</span>
      <strong>{{ challengeCorrectInRange }}题</strong>
    </div>
    <div class="challenge-card">
      <span class="challenge-label">累计</span>
      <strong>{{ challengeSolvedCount }}</strong>
    </div>
  </div>

  <div v-else class="quick-counts" aria-label="快捷数字">
    <button
      v-for="num in QUICK_COUNTS"
      :key="num"
      class="quick-chip"
      type="button"
      @click="applyQuickCount(num)"
    >
      {{ num }}
    </button>
  </div>
</div>
</section>

<section v-if="isChallengeMode" class="challenge-preview-shell">
  <div class="preview-copy">
    <Target :size="18" />
    <span>看球阵猜数字</span>
  </div>
</section>

```

```

    </div>
    <div class="challenge-ball-shell">
      <BallArray :count="challengeTargetCount" size="compact" />
    </div>
  </section>

  <section class="pad-stage">
    <NumberPad size="compact" @input="handleInput" @delete="handleDelete" @submit="handleSubmit" />
  </section>
</section>
</main>
</div>

<main v-else class="result-screen view-wrapper">
  <section class="result-number-shell">
    <div class="result-number-block">
      <div class="counter-badge result-badge">
        {{ isChallengeMode ? (challengeLastCorrect ? '挑战成功' : '答案公布') : '探索结果' }}
      </div>
      <div class="big-number font-number">{{ currentCount }}</div>
      <p v-if="isChallengeMode" class="result-copy">
        {{ challengeLastCorrect ? `你猜对了，答案就是 ${currentCount}。` : `你猜 ${challengeLastGuess}，正确答案是 ${currentCount}。` }}
      </p>
      <p v-else class="result-copy">{{ currentCount }} = {{ numberParts }}</p>
    </div>
  </section>

  <section class="result-ball-shell">
    <BallArray :count="currentCount" />
  </section>

  <section v-if="isChallengeMode" class="result-tools" aria-label="挑战结果操作">
    <div class="challenge-summary">
      <div class="challenge-card">
        <span class="challenge-label">当前范围</span>
        <strong>{{ currentChallengeRange.label }}</strong>
      </div>
      <div class="challenge-card">
        <span class="challenge-label">已答对</span>
        <strong>{{ challengeSolvedCount }}</strong>
      </div>
      <div class="challenge-card">
        <span class="challenge-label">本档训练</span>
        <strong>{{ challengeCorrectInRange }}题</strong>
      </div>
    </div>

    <button class="play-again-btn" @click="nextChallenge">
      <Target :size="20" />
      <span>下一题</span>
    </button>
  </section>

```

```

</button>

<button class="secondary-btn" type="button" @click="retryChallengeRange">
  重新练这一档
</button>
</section>

<section v-else class="result-tools" aria-label="连续探索操作">
  <div class="step-controls">
    <button class="step-btn" type="button" @click="stepCount(-1)">-1</button>
    <button class="step-btn" type="button" @click="stepCount(1)">+1</button>
    <button class="step-btn" type="button" @click="stepCount(10)">+10</button>
  </div>

  <div class="quick-counts result-counts" aria-label="常用数字">
    <button
      v-for="num in QUICK_COUNTS"
      :key="`result-${num}`"
      class="quick-chip"
      type="button"
      @click="jumpToCount(num)"
    >
      {{ num }}
    </button>
  </div>

  <button class="play-again-btn" @click="resetExplore">
    <RotateCcw :size="20" />
    <span>重新输入</span>
  </button>
</section>
</main>
</Transition>
</div>
</template>

<style scoped>
.explore-page {
  min-height: 100dvh;
  display: flex;
  flex-direction: column;
  padding-top: env(safe-area-inset-top);
  padding-bottom: calc(env(safe-area-inset-bottom) + 12px);
  background:
    radial-gradient(circle at top, rgba(92, 157, 255, 0.12) 0%, rgba(92, 157, 255, 0) 38%),
    linear-gradient(180deg, #f8fbff 0%, #edf4ff 100%);
  overflow: hidden;
}

.top-bar {

```



```
display: flex;
align-items: center;
padding: 12px 16px 4px;
}
```

```
.back-btn {
width: 48px;
height: 48px;
border: 1px solid rgba(92, 157, 255, 0.12);
border-radius: 18px;
background: rgba(255, 255, 255, 0.88);
box-shadow: 0 12px 28px rgba(49, 120, 246, 0.12);
display: inline-flex;
align-items: center;
justify-content: center;
color: var(--brand-primary);
backdrop-filter: blur(12px);
-webkit-backdrop-filter: blur(12px);
transition: transform var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard), box-shadow var(--duration-fast) var(--ease-standard);
}
```

```
.back-btn:focus-visible,
.play-again-btn:focus-visible,
.quick-chip:focus-visible,
.step-btn:focus-visible,
.mode-btn:focus-visible,
.secondary-btn:focus-visible,
.range-chip:focus-visible {
outline: none;
box-shadow: 0 0 0 4px rgba(92, 157, 255, 0.18);
}
```

```
.back-btn:active,
.play-again-btn:active {
transform: scale(0.95);
}
```

```
.view-wrapper {
flex: 1;
display: flex;
flex-direction: column;
}
```

```
.input-screen,
.result-screen {
flex: 1;
min-height: 0;
display: flex;
flex-direction: column;
padding: 8px 14px 0;
```

```
}
```

```
.input-screen {  
  gap: 12px;  
}
```

```
.result-screen {  
  gap: 12px;  
  padding-inline: 10px;  
  padding-bottom: calc(env(safe-area-inset-bottom) + 12px);  
  overflow-y: auto;  
  -webkit-overflow-scrolling: touch;  
}
```

```
.mode-switch {  
  display: grid;  
  grid-template-columns: repeat(2, minmax(0, 1fr));  
  gap: 10px;  
}
```

```
.mode-btn,  
.secondary-btn {  
  min-height: 48px;  
  border: 1px solid rgba(92, 157, 255, 0.14);  
  border-radius: 18px;  
  background: rgba(255, 255, 255, 0.84);  
  color: var(--text-blue);  
  font-size: 15px;  
  font-weight: 800;  
  transition: transform var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard);  
}
```

```
.mode-btn.is-active {  
  background: linear-gradient(180deg, #6da8ff 0%, #4b86f3 100%);  
  border-color: rgba(75, 134, 243, 0.24);  
  color: #fff;  
  box-shadow: 0 12px 22px rgba(75, 134, 243, 0.18);  
}
```

```
.range-selector-shell,  
.challenge-workspace {  
  border: 1px solid rgba(92, 157, 255, 0.12);  
  border-radius: 30px;  
  background: rgba(255, 255, 255, 0.9);  
  box-shadow: 0 16px 30px rgba(58, 87, 152, 0.08);  
  backdrop-filter: blur(14px);  
  -webkit-backdrop-filter: blur(14px);  
}
```

```
.range-selector-shell {
```

```
padding: 10px 12px;
}
```

```
.range-selector-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 8px;
}
```

```
.range-title-badge {
  flex-shrink: 0;
}
```

```
.range-selector-copy {
  margin: 0;
  color: var(--text-blue-light);
  font-size: 12px;
  font-weight: 700;
  text-align: right;
}
```

```
.challenge-workspace {
  padding: 8px;
  display: flex;
  flex-direction: column;
  gap: 8px;
}
```

```
.explore-workspace {
  display: flex;
  flex-direction: column;
  gap: 12px;
}
```

```
.number-stage,
.result-number-shell,
.result-ball-shell,
.play-again-btn,
.step-btn,
.challenge-preview-shell {
  position: relative;
  border: 1px solid rgba(92, 157, 255, 0.12);
  background: rgba(255, 255, 255, 0.88);
  box-shadow: 0 16px 30px rgba(58, 87, 152, 0.08);
  backdrop-filter: blur(14px);
  -webkit-backdrop-filter: blur(14px);
}
```

```
.number-stage,  
.result-number-shell,  
.challenge-preview-shell {  
  overflow: hidden;  
  border-radius: 34px;  
}
```

```
.number-stage {  
  padding: 10px 12px 12px;  
  background: rgba(255, 255, 255, 0.96);  
  border: 1px solid rgba(92, 157, 255, 0.14);  
  box-shadow: 0 20px 36px rgba(58, 87, 152, 0.1);  
}
```

```
.number-card {  
  width: 100%;  
}
```

```
.number-card-top,  
.number-display {  
  display: flex;  
  align-items: center;  
}
```

```
.number-card-top {  
  justify-content: flex-start;  
  gap: 8px;  
  margin-bottom: 8px;  
}
```

```
.counter-badge {  
  padding: 8px 12px;  
  border-radius: var(--radius-full);  
  background: rgba(92, 157, 255, 0.1);  
  border: 1px solid rgba(92, 157, 255, 0.12);  
  color: var(--brand-primary);  
  font-size: var(--font-sm);  
  font-weight: 800;  
}
```

```
.range-badge {  
  background: rgba(107, 203, 119, 0.12);  
  border-color: rgba(107, 203, 119, 0.18);  
  color: var(--text-teal);  
}
```

```
.number-display {  
  justify-content: center;  
  min-height: 76px;  
}
```

```
.big-number {
  position: relative;
  z-index: 1;
  min-width: clamp(88px, 24vw, 132px);
  padding: 12px 14px;
  border-radius: var(--radius-lg);
  border: 2px solid rgba(92, 157, 255, 0.18);
  background: linear-gradient(180deg, #ffffff 0%, #f1f7ff 100%);
  box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.92), 0 10px 20px rgba(92, 157, 255, 0.08);
  color: var(--text-blue-dark);
  font-size: clamp(48px, 14vw, 76px);
  font-weight: 800;
  line-height: 1;
  text-align: center;
  transition: transform var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard), box-shadow var(--duration-fast) var(--ease-standard);
}

.big-number.is-empty {
  color: var(--text-muted);
}

.helper-copy,
.status-copy,
.result-copy {
  margin: 0;
  text-align: center;
}

.helper-copy {
  margin-top: 8px;
  color: var(--text-secondary);
  font-size: 14px;
  line-height: 1.5;
}

.status-copy {
  min-height: 16px;
  margin-top: 6px;
  color: var(--text-blue-light);
  font-size: 12px;
  font-weight: 700;
  line-height: 1.25;
}

.status-copy.is-warning {
  color: var(--text-warning);
}

.status-copy.is-success {
```

```
    color: var(--text-success);
}

.challenge-meta,
.challenge-summary {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 6px;
  margin-top: 8px;
}

.range-selector {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  gap: 6px;
  margin-top: 2px;
}

.range-chip {
  min-height: 44px;
  padding: 0 12px;
  border: 1px solid rgba(92, 157, 255, 0.16);
  border-radius: 999px;
  background: rgba(255, 255, 255, 0.9);
  color: var(--text-blue);
  font-size: 13px;
  font-weight: 800;
  transition: transform var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard), color var(--duration-fast) var(--ease-standard);
}

.range-chip.is-active {
  background: linear-gradient(180deg, #6ecb96 0%, #4aac70 100%);
  border-color: rgba(74, 172, 112, 0.22);
  color: #fff;
  box-shadow: 0 10px 18px rgba(74, 172, 112, 0.16);
}

.challenge-card {
  min-height: 44px;
  padding: 6px 4px;
  border: 1px solid rgba(92, 157, 255, 0.12);
  border-radius: 14px;
  background: rgba(255, 255, 255, 0.82);
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 2px;
  text-align: center;
```

```
}
```

```
.challenge-label {  
  color: var(--text-blue-light);  
  font-size: 10px;  
  font-weight: 700;  
}
```

```
.challenge-card strong {  
  color: var(--text-blue-dark);  
  font-size: 13px;  
  font-weight: 900;  
}
```

```
.quick-counts {  
  display: flex;  
  flex-wrap: wrap;  
  justify-content: center;  
  gap: 8px;  
  margin-top: 14px;  
}
```

```
.quick-chip {  
  min-height: 44px;  
  padding: 0 14px;  
  border: 1px solid rgba(92, 157, 255, 0.16);  
  border-radius: 999px;  
  background: rgba(255, 255, 255, 0.9);  
  color: var(--brand-primary);  
  font-size: 14px;  
  font-weight: 800;  
  transition: transform var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard);  
}
```

```
.quick-chip:active,  
.step-btn:active,  
.mode-btn:active,  
.secondary-btn:active,  
.range-chip:active {  
  transform: scale(0.96);  
}
```

```
.number-stage.is-shaking .big-number {  
  border-color: rgba(255, 107, 107, 0.4);  
  animation: toy-shake 0.28s ease;  
}
```

```
.number-stage.is-shaking {  
  animation: none;  
}
```

```
.challenge-preview-shell {  
  padding: 8px;  
  border-radius: 22px;  
  background: linear-gradient(180deg, rgba(247, 250, 255, 0.98) 0%, rgba(235, 243, 255, 0.95) 100%);  
}
```

```
.preview-copy {  
  display: inline-flex;  
  align-items: center;  
  gap: 8px;  
  color: var(--text-blue);  
  font-size: 12px;  
  font-weight: 800;  
  margin-bottom: 6px;  
}
```

```
.challenge-ball-shell {  
  border-radius: 20px;  
  overflow: hidden;  
}
```

```
.result-number-shell {  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  padding: 18px 18px 16px;  
  background: linear-gradient(180deg, rgba(255, 255, 255, 0.98) 0%, rgba(241, 247, 255, 0.96) 100%);  
}
```

```
.result-number-block {  
  width: 100%;  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
  gap: 10px;  
}
```

```
.result-badge {  
  background: rgba(107, 203, 119, 0.12);  
  border-color: rgba(107, 203, 119, 0.18);  
  color: var(--text-teal);  
}
```

```
.result-copy {  
  color: var(--text-blue-light);  
  font-size: 15px;  
  font-weight: 800;  
  line-height: 1.5;  
}
```



```
.result-ball-shell {
  width: 100%;
  max-width: min(100%, calc(100dvh - 320px));
  aspect-ratio: 1 / 1;
  margin: 0 auto;
  border-radius: 34px;
  padding: 4px;
  overflow: hidden;
}

.result-tools {
  display: flex;
  flex-direction: column;
  gap: 12px;
}

.step-controls {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 10px;
}

.step-btn {
  min-height: 52px;
  border-radius: 20px;
  background: linear-gradient(180deg, rgba(255, 255, 255, 0.98) 0%, rgba(242, 247, 255, 0.96) 100%);
  color: var(--text-blue-dark);
  font-size: 18px;
  font-weight: 900;
  transition: transform var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard);
}

.result-counts {
  margin-top: 0;
}

.play-again-btn {
  width: 100%;
  min-height: 64px;
  border-radius: 24px;
  padding: 0 20px;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 10px;
  color: #fff;
  font-size: 22px;
  font-weight: 900;
  background: linear-gradient(180deg, #62c98a 0%, #48aa70 100%);
}
```

```
border-color: rgba(74, 172, 112, 0.24);
box-shadow: 0 12px 26px rgba(74, 172, 112, 0.2);
}

.secondary-btn {
  color: var(--text-blue-light);
}

@media (hover: hover) {
  .quick-chip:hover,
  .step-btn:hover,
  .secondary-btn:hover,
  .range-chip:hover:not(.is-active) {
    border-color: rgba(92, 157, 255, 0.28);
    background: var(--bg-light);
  }

  .mode-btn:hover:not(.is-active) {
    background: rgba(255, 255, 255, 0.94);
  }

  .play-again-btn:hover {
    box-shadow: 0 16px 30px rgba(74, 172, 112, 0.24);
  }
}

.fade-enter-active,
.fade-leave-active {
  transition: opacity 0.35s ease, transform 0.35s ease;
}

.fade-enter-from,
.fade-leave-to {
  opacity: 0;
  transform: scale(0.96) translateY(20px);
}

@media (max-width: 420px) {
  .range-selector-header {
    flex-direction: column;
    align-items: flex-start;
  }

  .range-selector-copy {
    text-align: left;
  }

  .number-stage {
    padding: 8px 10px 10px;
  }
}
```

```
.number-card-top {
  margin-bottom: 8px;
}

.number-display {
  min-height: 66px;
}

.big-number {
  min-width: 72px;
  padding: 8px 10px;
}

.challenge-meta,
.challenge-summary {
  grid-template-columns: repeat(3, minmax(0, 1fr));
}

.challenge-workspace {
  padding: 6px;
  gap: 6px;
}

.result-copy {
  font-size: 14px;
}

.step-controls {
  gap: 8px;
}

.step-btn {
  min-height: 48px;
  border-radius: 18px;
  font-size: 17px;
}
}

@media (max-width: 360px) {
  .counter-badge {
    padding: 8px 10px;
    font-size: 12px;
  }

  .number-display {
    min-height: 92px;
  }

  .big-number {
```

```
        font-size: 42px;
    }
}

@media (max-width: 959px) and (max-height: 860px) {
    .number-stage {
        padding-top: 8px;
        padding-bottom: 8px;
    }

    .number-display {
        min-height: 88px;
    }

    .result-ball-shell {
        max-width: min(100%, calc(100dvh - 300px));
    }
}

@media (min-width: 768px) {
    .number-stage {
        padding: 14px 18px 18px;
    }
}
</style>
```